

pwnkit: Methodology-Aware Evaluation and Cost-Aware Triage for Autonomous Pentesting Agents

PwnKit Labs
github.com/PwnKit-Labs/pwnkit

Abstract

Autonomous pentesting agents are frequently summarized by a single benchmark headline, although reported performance is sensitive to retry protocol, benchmark substrate, model/runtime, turn budget, and evidence policy. We present pwnkit, an open-source shell-first pentesting framework with layered triage and blind verification, and pair it with a methodology-aware reporting protocol that separates retained artifact-backed evidence from historical mixed publication lines. In the current public ledger snapshot (2026-04-10), pwnkit reports a retained artifact-backed XBOW aggregate of 99/104. A 21-run triage ablation (2026-04-11) shows mode-dependent behavior: full-moat triage is a precision/recall tradeoff in white-box slices, a strict efficiency improvement in black-box slices, and weakly additive on npm-bench over default scaffolding. The primary contribution is a combined systems-and-methodology perspective in which evidence lineage and protocol disclosure are treated as first-class engineering surfaces for non-deterministic security agents.

1 Introduction

Single benchmark percentages for autonomous security agents hide variance and protocol sensitivity. In practice, score changes can come from retry strategy, benchmark fork, or evidence accounting policy rather than meaningful capability changes. We frame pwnkit as both:

- a **systems artifact**: shell-first exploitation, layered triage, blind verification, and multi-target operation;
- a **methodology artifact**: protocol-explicit benchmark reporting, retained-evidence lineage, and slice-aware interpretation.

Contributions.

1. A reproducible benchmarking protocol for non-deterministic pentesting agents (repeat framing, uncertainty, cost ceilings).
2. A machine-auditable ledger model separating retained artifact-backed and historical mixed publication lines.
3. A productionized multi-layer triage stack integrated in a live scanner pipeline, with per-layer verdict telemetry.
4. A mode-dependent ablation narrative reporting both positive and negative outcomes, avoiding single-line “moat works” claims.
5. A learned-routing foundation for per-finding policy selection.

2 System Overview

The pipeline is:

Plan → Discover → Attack → Triage → Verify → Report

For web targets, pwnkit uses shell-first execution (`bash` as primary tool), with structured helpers optional. Findings then pass through layered triage and verification before final reporting.

Core implementation paths include scanner orchestration, unified pipeline execution, and triage modules in the core package.

3 Benchmark Methodology

We distinguish three reporting regimes:

- single-shot,
- best-of-K,
- repeat-N per-attempt success rate with 95% Wilson CI.

The benchmark ledger tracks:

- retained artifact-backed totals (machine-reconstructible),
- historical mixed local+CI publication totals (legacy publication line).

This separation is implemented in `packages/benchmark/results/benchmark-ledger.json` and reflected in the docs benchmark surface.

3.1 Current Ledger Snapshot (2026-04-10)

Surface	Black-box	White-box	Aggregate
Retained artifact-backed	74/104	79/104	99/104
Historical mixed publication	90/104	5 (white-box-only)	95/104

Table 1: Current ledger surfaces are intentionally separated and should not be merged.

4 Triage Stack and Dynamic Routing

The triage stack combines deterministic filters, feature extraction, reachability and oracle checks, PoV/structured verify layers, memory-based suppression, and optional debate/tree-search layers. Per-layer telemetry is attached to findings, creating supervision data for learned per-finding routing.

Dynamic routing motivation is empirical: no static profile dominates across white-box, black-box, and package slices.

Profile	Flags	Findings	Cost (USD)	USD/flag
<i>XBOW white-box (limit=50)</i>				
none	43/50	67	14.34	0.33
no-triage	44/50	67	17.17	0.39
moat-only	41/50	25	26.89	0.66
moat	41/50	25	21.82	0.53
<i>XBOW black-box (limit=25)</i>				
none	18/25	27	13.72	0.76
no-triage	19/25	34	10.37	0.55
moat-only	18/25	13	11.22	0.62
moat	19/25	14	10.04	0.53

Table 2: Ablation snapshot showing mode-dependent moat behavior.

Profile	F1	TPR	FPR	Safe correct
none	0.973	1.00	0.11	24/27
no-triage	0.964	1.00	0.15	23/27
moat-only	0.964	1.00	0.15	23/27
moat	0.956	1.00	0.19	22/27
default	0.956	1.00	0.19	22/27

Table 3: npm-bench profile outcomes (81 packages) from the same ablation set.

4.1 Ablation Snapshot (2026-04-11)

5 Empirical Highlights

Current repository-reported posture includes:

- retained artifact-backed XBOW aggregate at 99/104,
- separate black-box and white-box lines,
- a 21-run triage ablation matrix with mode-dependent outcomes.

Key methodological outcome: no static profile dominates all slices, so policy selection should be context-aware.

5.1 Dataset and router artifacts

Current intermediate artifacts include:

- triage dataset v1 stats: 969 samples,
- router v2 metadata: 1514 samples, 55 features, 5-fold CV mean F1 $\approx$ 0.962.

These are treated as offline modeling artifacts, not final deployment claims.

6 Evaluation Details and Interpretation

This draft intentionally reports per-slice outcomes rather than collapsing all settings into one global number.

6.1 White-box versus black-box behavior

The ablation in Table 2 shows that the same triage profile can have different utility depending on evidence availability. In white-box mode, source access increases the attack agent’s direct exploit signal and some expensive triage layers can over-prune, producing a precision/recall tradeoff. In black-box mode, where candidate findings are noisier, additional verification layers provide clearer net benefit on findings efficiency and USD/flag.

6.2 Package slice caveat

Table 3 shows behavior distinct from XBOW slices. The current interpretation is that profile effects are entangled with scan scaffolding and coarse package-level labels. This motivates treating package evaluation as a separate operating regime rather than extrapolating web-target behavior.

6.3 Layer-level failure mode

Single-layer isolation runs reported in the project docs identify one recurring negative outlier (‘egats’) on hard slices, while other layers are neutral or positive in that context. This supports a policy of feature-gated layer rollout and per-layer telemetry before broad default enablement.

7 Discussion

7.1 Why methodology is part of the contribution

For stochastic, tool-using security agents, benchmarking is not just post-processing. Protocol choices alter reported results materially, so measurement design and evidence accounting are engineering surfaces that deserve explicit implementation and review.

7.2 Operational policy implication

The measured outcome is not "enable all layers always." A practical policy is:

1. keep deterministic/cheap layers broadly enabled,
2. gate expensive layers by context,
3. track per-layer impact over time,
4. promote only slice-robust defaults.

7.3 From static profiles to routing

The router path in current artifacts should be understood as an optimization framework rather than a completed deployment claim. The offline metrics are promising, but production value should be established by online A/B outcomes on explicitly frozen benchmark protocols.

8 Related Work

Current pwnkit docs position this work adjacent to three threads:

- autonomous pentesting systems with differing orchestration styles (single-agent, role-based, and multi-agent workflows),
- triage pipelines combining deterministic checks and model-based judgments,
- hybrid feature+model signals for vulnerability classification and filtering.

The comparative caveat is central: cross-project benchmark numbers are only loosely comparable unless protocol and substrate are aligned.

Representative systems in the current pwnkit docs include BoxPwnr, Shannon, KinoSec, Cyber-AutoAgent, deadend-cli, and MAPTA[6, 3, 4, 2, 9, 1]. For FP-triage architecture context, pwnkit docs reference Endor Labs and Semgrep Assistant disclosures[5, 8]. For hybrid feature+model signals, pwnkit docs use VulnBERT-style framing as design inspiration rather than a direct task-equivalent claim[7].

Table 4 collects the self-reported XBOW aggregates for these systems. The numbers come from each project’s own public disclosures, not from a common evaluation run on matched hardware, model, or harness; they are included for rough positioning, not as a leaderboard claim.

System	Aggregate	Model / Approach	Notes
BoxPwnr[6]	97.1% (101/104)	Best-of-N, 10 configs	Context compaction, loop detection
Shannon[3]	96.2% (100/104)	White-box, multi-agent	Source-to-sink taint
pwnkit (this work)	95.2% (99/104)	Shell-first, gpt-5.4	Retained artifact-backed; see Table 1
KinoSec[4]	92.3% (96/104)	Black-box HTTP only	50-turn hard cap
Cyber-AutoAgent[2]	84.6% (88/104)	Meta-agent	Self-rewriting prompts
deadend-cli[9]	77.6% (~76/98)	Single-agent CLI	ADaPT decomposition
MAPTA[1]	76.9% (80/104)	3-role multi-agent	Evidence-gated branching

Table 4: Self-reported XBOW aggregates across representative autonomous pentesting systems. Cross-project numbers are protocol-sensitive (differing model, harness, retry policy, challenge subset) and should be interpreted as rough positioning rather than a matched-conditions leaderboard.

9 Limitations and Threats to Validity

We highlight benchmark dependence, run-to-run variance, slice sensitivity, documentation and implementation drift risk, and cost-latency tradeoffs in deeper triage pipelines.

Additional caveats:

- package-level supervision can be coarse for per-finding labels;
- leaderboard comparisons across projects are protocol-sensitive and should be interpreted with caution;
- rapidly evolving repos can create temporary doc/code mismatch.

10 Claim Traceability Checklist

Before submission freeze, each numerical claim should be tied to:

1. an as-of date,

2. an artifact path,
3. protocol context (retry regime, budget, model/runtime),
4. whether the claim is retained artifact-backed or historical mixed.

The companion files `docs/paper/evaluation.md` and `docs/paper/related_work.md` exist to keep this audit step lightweight.

11 Reproducibility Notes

Canonical evidence files and runners are in:

- `docs/src/content/docs/methodology.md`
- `docs/src/content/docs/benchmark.md`
- `docs/src/content/docs/research/2026-04-11-ablation.md`
- `docs/src/content/docs/research/fp-reduction-moat.md`
- `packages/benchmark/results/benchmark-ledger.json`
- `packages/benchmark/results/triage-dataset-v1.stats.json`
- `packages/benchmark/results/triage-router-v2-meta.json`
- `packages/benchmark/src/xbow-runner.ts`
- `packages/benchmark/src/wilson.ts`

Companion drafting notes:

- `docs/paper/pwnkit.md`
- `docs/paper/evaluation.md`
- `docs/paper/related_work.md`

12 Conclusion

For autonomous pentesting agents, measurement protocol quality is part of the core contribution. pwnkit's current evidence suggests that transparent protocol disclosure, retained-evidence lineage, and mode-aware triage policy are more reliable than single headline percentages.

References

- [1] MAPTA Authors. Mapta. arXiv preprint arXiv:2508.20816, 2025. URL <https://arxiv.org/abs/2508.20816>.

- [2] Cyber-AutoAgent Authors. Cyber-autoagent: From single-agent to meta-agent, building the leading open-source autonomous cyber agent. Data Science Collective (Medium), 2026. URL <https://medium.com/data-science-collective/from-single-agent-to-meta-agent-building-the-leading-open-source-autonomous-cyber-agent-e1b7>. Accessed 2026-04-17.
- [3] Keygraph. Shannon. GitHub repository, 2026. URL <https://github.com/KeygraphHQ/shannon>. Accessed 2026-04-17.
- [4] KinoSec. Kinosec. Product website, 2026. URL <https://kinosec.ai>. Accessed 2026-04-17.
- [5] Endor Labs. Endor labs ai sast materials. Vendor technical materials, 2026. URL <https://www.endorlabs.com/>. Accessed 2026-04-17.
- [6] Francisco Oca. Boxpwnr. GitHub repository, 2026. URL <https://github.com/Oca/BoxPwnr>. Accessed 2026-04-17.
- [7] Guanni Qu. Vulnbert writeup. Pebblebed blog, 2026. URL <https://pebblebed.com/blog/kernel-bugs>. Accessed 2026-04-17.
- [8] Semgrep. Semgrep assistant materials. Vendor technical materials, 2026. URL <https://semgrep.dev/>. Accessed 2026-04-17.
- [9] xoxruns. Feedback-driven iteration and fully local webapp pentesting ai agent achieving 78 on xbow. Medium post, 2026. Published online, accessed 2026-04-17.